Отчёта по лабораторной работе 3

Агентное Моделирование

Курилко-Рюмин Евгений Михайлович

Содержание

Список иллюстраций

Список таблиц

1 Цель работы

Изучить основы агентного моделирования (Agent-Based Modeling, ABM) на примере модели Daisyworld, реализованной с использованием фреймворка Agents.jl в языке Julia. Освоить принципы эмерджентности, автономности агентов и саморегуляции системы.

2 Задание

1. Создать рабочий каталог проекта DrWatson для размещения кода модели Daisyworld.
2. Установить необходимые Julia-пакеты (Agents, Plots, DrWatson, Literate и др.).
3. Реализовать модель Daisyworld на основе фреймворка Agents.jl.
4. Выполнить визуализацию состояния модели в виде тепловых карт на шагах 0, 5, 40.
5. Создать анимацию динамики модели.
6. Построить графики динамики численности маргариток.
7. Реализовать сценарий изменения солнечной светимости (luminosity ramp) и построить трёхпанельные графики полной динамики.
8. Выполнить параметрическое исследование с варьированием начальной доли белых маргариток (init_white) и максимального возраста (max_age).
9. Преобразовать рабочий код в литературный стиль с разметкой Literate.jl.
10. Сгенерировать из литературного кода три производных формата: чистый .jl-скрипт, Jupyter notebook .ipynb, Markdown-документацию .qmd.

3 Теоретическое введение

Агентное моделирование (Agent-Based Modeling, ABM) — это вычислительный метод исследования сложных систем, в котором поведение системы возникает из взаимодействия множества автономных сущностей, называемых агентами. Вместо описания системы глобальными уравнениями, моделируется каждая индивидуальная единица и правила её поведения.

Ключевые принципы ABM:

- **Эмерджентность** — глобальное поведение системы возникает из локальных взаимодействий агентов и не задаётся явно.
- **Автономность** — каждый агент действует независимо, на основе своих правил и локальной информации.
- **Гетерогенность** — агенты могут различаться по своим характеристикам и правилам поведения.
- **Локальность** — агенты взаимодействуют только с ближайшими соседями.

3.1 Модель Daisyworld

Модель Daisyworld была предложена Джеймсом Лавлоком и Эндрю Уотсоном в 1983 году как упрощённая демонстрация гипотезы Геи — идеи о том, что биосфера Земли может саморегулироваться, поддерживая условия, благоприятные для жизни.

Компоненты модели:

- **Агенты:** маргаритки двух видов — чёрные (низкое альбедо, поглощают тепло) и белые (высокое альбедо, отражают тепло).
- **Среда:** двумерная решётка размером 30×30 клеток.
- **Динамика:** маргаритки размножаются на соседние пустые клетки с вероятностью, зависящей от локальной температуры. Стареют и умирают по достижении максимального возраста.
- **Саморегуляция:** при повышении температуры белые маргаритки получают преимущество (охлаждают планету), при понижении — чёрные (нагревают планету).

4 Выполнение лабораторной работы

4.1 Настройка окружения

4.1.1 Запуск Julia и загрузка DrWatson

Запускаем интерпретатор Julia

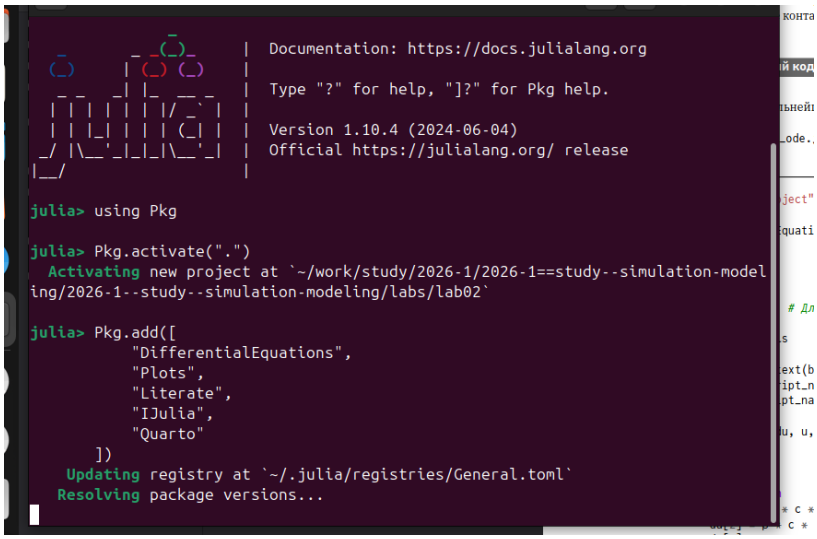

Рисунок 4.1: Запускаем Julia

Затем подключаем пакет DrWatson, предназначенный для организации воспроизводимых научных вычислений. DrWatson предоставляет стандартную структуру каталогов проекта и удобные функции навигации (`plotsdir()`, `scriptsdir()`, `srcdir()`).

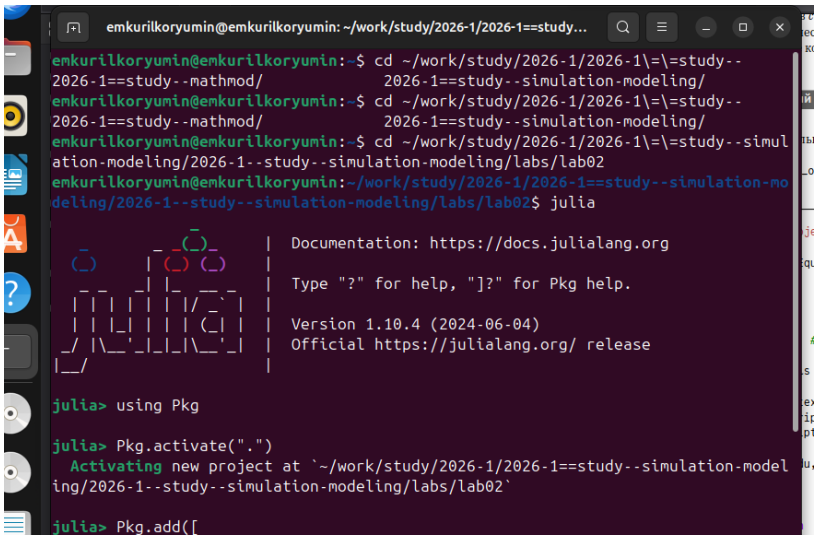

Рисунок 4.2: Загрузка пакета DrWatson

4.1.2 Создание проекта DrWatson

Инициализируем проект с помощью DrWatson который автоматически создает каталоги scripts/, src/, data/, plots/ и другие.

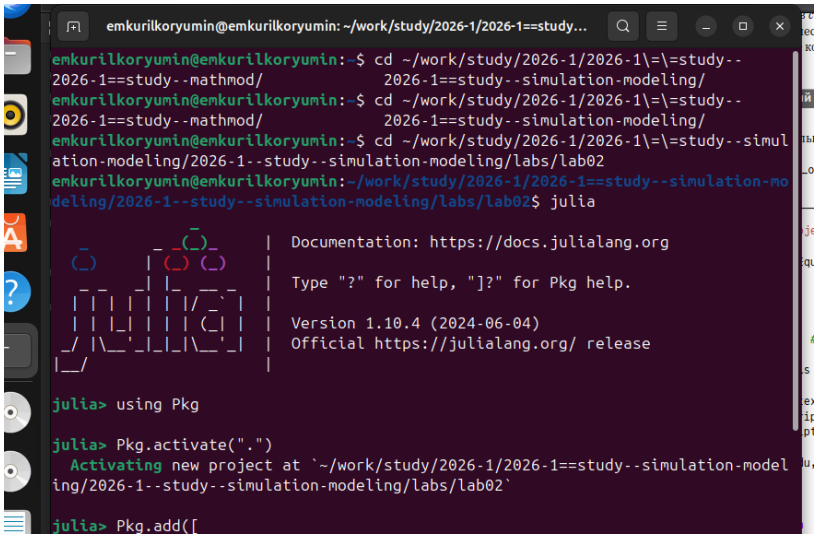

Рисунок 4.3: Создание проекта

###Установка пакетов

Устанавливаем пакеты путем применения следующий команды

```julia
using Pkg
Pkg.add([
        "DrWatson","Agents","CairoMakie"
         "Literate","IJulia",
])
```

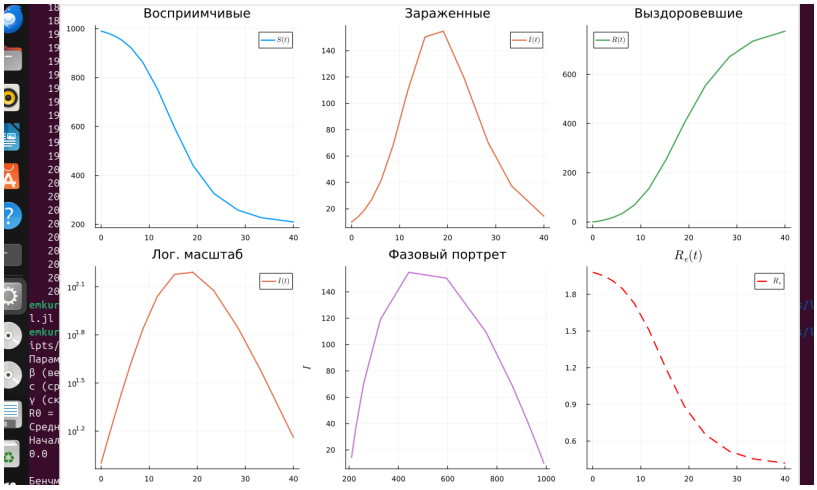

Рисунок 4.4: Установленые пакеты

4.2 Модель Daisyworld

Основной код модели размещён в файле `src/daisyworld.jl`. Модель реализована на основе фреймворка Agents

4.2.1 Определение агента

Каждая маргаритка описывается структурой `Daisy`, содержащей идентификатор, позицию на решётке, породу (`:black` или `:white`) и возраст:

```julia
@agent struct Daisy(GridAgent{2})
    breed::Symbol
    age::Int
```

```
    albedo::Float64
end
```

4.2.2 Инициализация модели

Функция инициализации создаёт решётку, случайно размещает маргаритки заданных долей и рассчитывает начальные температуры:

```
function daisyworld(;
    gridsize = (30, 30),
    init_black = 0.2,
    init_white = 0.2,
    albedo_black = 0.25,
    albedo_white = 0.75,
    surface_albedo = 0.4,
    solar_luminosity = 1.0,
    solar_change = 0.005,
    max_age = 25,
    scenario = :default,
    seed = 165,
)
```

4.2.3 Шаг агента

На каждом шаге симуляции каждая маргаритка:

1. Увеличивает свой возраст на 1.
2. Если возраст превышает `max_age` — погибает, клетка освобождается.
3. Иначе — пытается произвести потомство на случайную соседнюю пустую клетку:

- вычисляется локальная температура по формуле (**?@eq-t-local**);
- вычисляется порог прорастания по формуле (**?@eq-seed-threshold**);
- если $p > 0$ и случайное число меньше p, создаётся новая маргаритка.

```
function daisy_step!(agent, model)
    agent.age += 1
    if agent.age >= model.max_age
        remove_agent!(agent, model)
        model.temperature[agent.pos...] = calc_temperature(
            model.surface_albedo, model
        )
        return
    end
    ## [illegible] [illegible]
    empty_neighbors = collect(
        nearby_positions(agent, model, 1; filter=isempty)
    )
    if !isempty(empty_neighbors)
        T = model.temperature[agent.pos...]
        seed_threshold = 0.1457 * T -
            0.0032 * T^2 - 0.6443
        if seed_threshold > 0 &&
                rand(abmrng(model)) < seed_threshold
            pos = rand(abmrng(model), empty_neighbors)
            add_agent!(pos, Daisy, model,
                agent.breed, 0, agent.albedo)
            model.temperature[pos...] =
                calc_temperature(agent.albedo, model)
        end
```

```
    end
end
```

4.2.4 Шаг модели

На каждом шаге модели обновляется солнечная светимость (в сценарии `ramp`) и пересчитываются температуры всех клеток:

```
function model_step!(model)
    if model.scenario == :ramp
        model.solar_luminosity += model.solar_change
        if model.solar_luminosity >= 2.0 ||
                model.solar_luminosity <= 0.5
            model.solar_change = -model.solar_change
        end
    end
    for pos in positions(model)
        if isempty(pos, model)
            model.temperature[pos...] = calc_temperature(
                model.surface_albedo, model
            )
        end
    end
end
```

##Визуализация модели (Базовая)

Скрипрт scripts/daisyworld.jl является базовой визуализацией состояния модели в виде тепловых карт. Скрипт создает модель с папаметрами и строит тепловую карту,где цвет клетки определяется типом маргаритки: чёрные маргаритки отоб-

ражаются тёмным цветом, белые — светлым, пустые клетки — промежуточным оттенком.

```julia
using DrWatson
@quickactivate "project"
using Agents
include(srcdir("daisyworld.jl"))
using CairoMakie

model = daisyworld()
for step in [0, 1, 5, 40]
    run!(model, step)
    fig = plot_daisyworld(model)
    fname = "daisy_step$(lpad(step, 3, '0')).png"
    save(plotsdir(fname), fig)
end
```

На изображенеии ниже представлен запуск скрипта

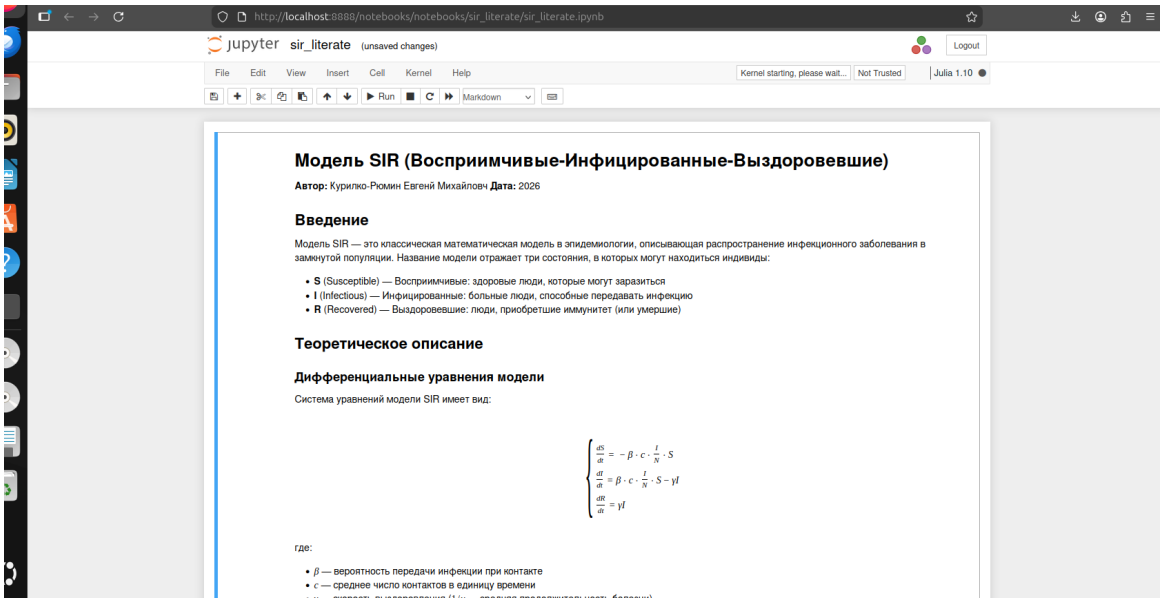

Рисунок 4.5: Запуск скрипта daisyworld.jl